

Two Castings Got Through

A pretrained AlexNet, fine-tuned on 6,633 casting images and tested on 715 it had never seen, classified 713 correctly. Both errors were defects called good. Nothing good was wrongly scrapped.

accuracy 99.72% macro F1 0.997 epochs 15 train time 6m 09s device CUDA

RESULT

The errors are asymmetric, and that asymmetry is the finding.

PIECES INSPECTED	ACCURACY	ACTUALLY DAMAGED	MISSED DAMAGED	FALSE ALARMS
715 held-out set	99.72% 713 of 715	63.36% 453 pieces	2 0.44% of defects	0 0.0% of good parts

	PRED. DEFECTIVE	PRED. GOOD
TRUE DEFECTIVE	451	2
TRUE GOOD	0	262

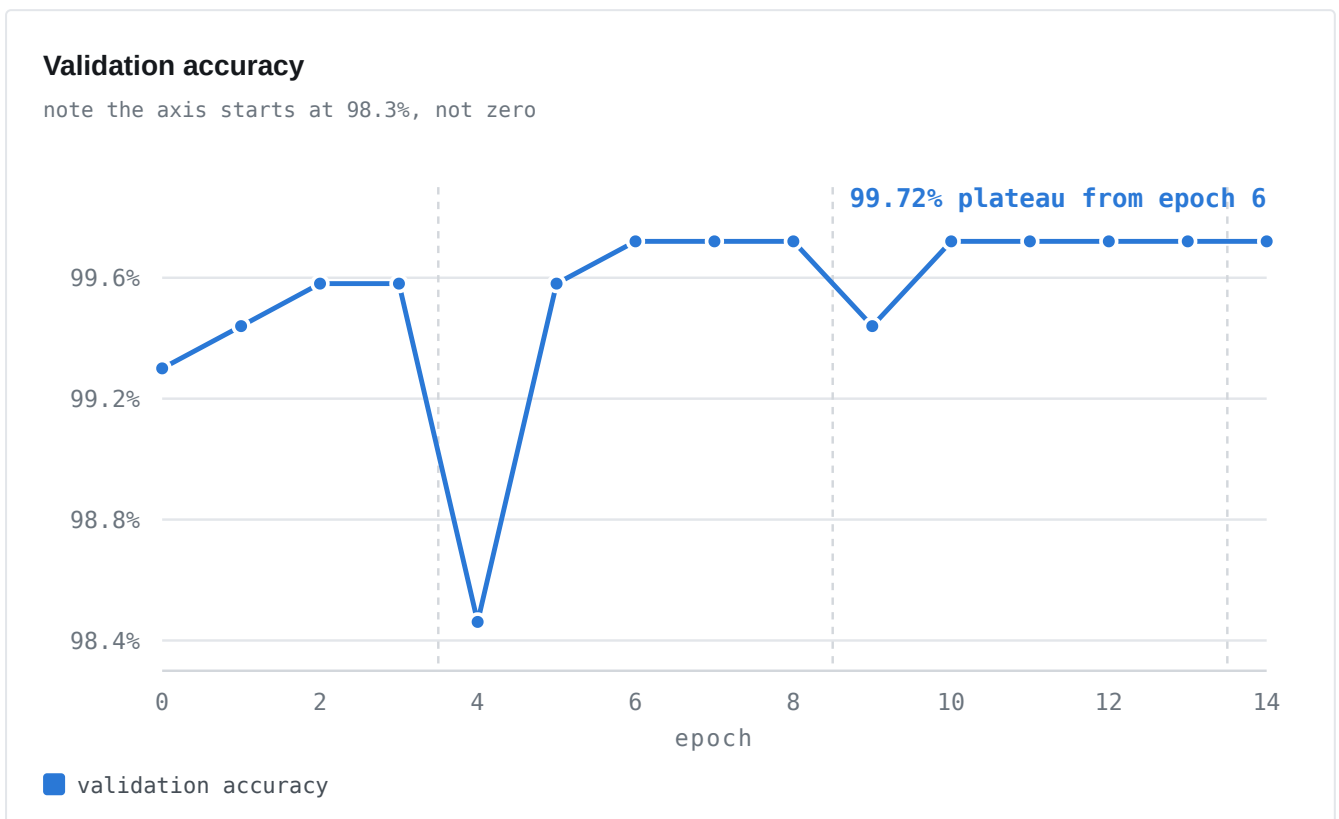
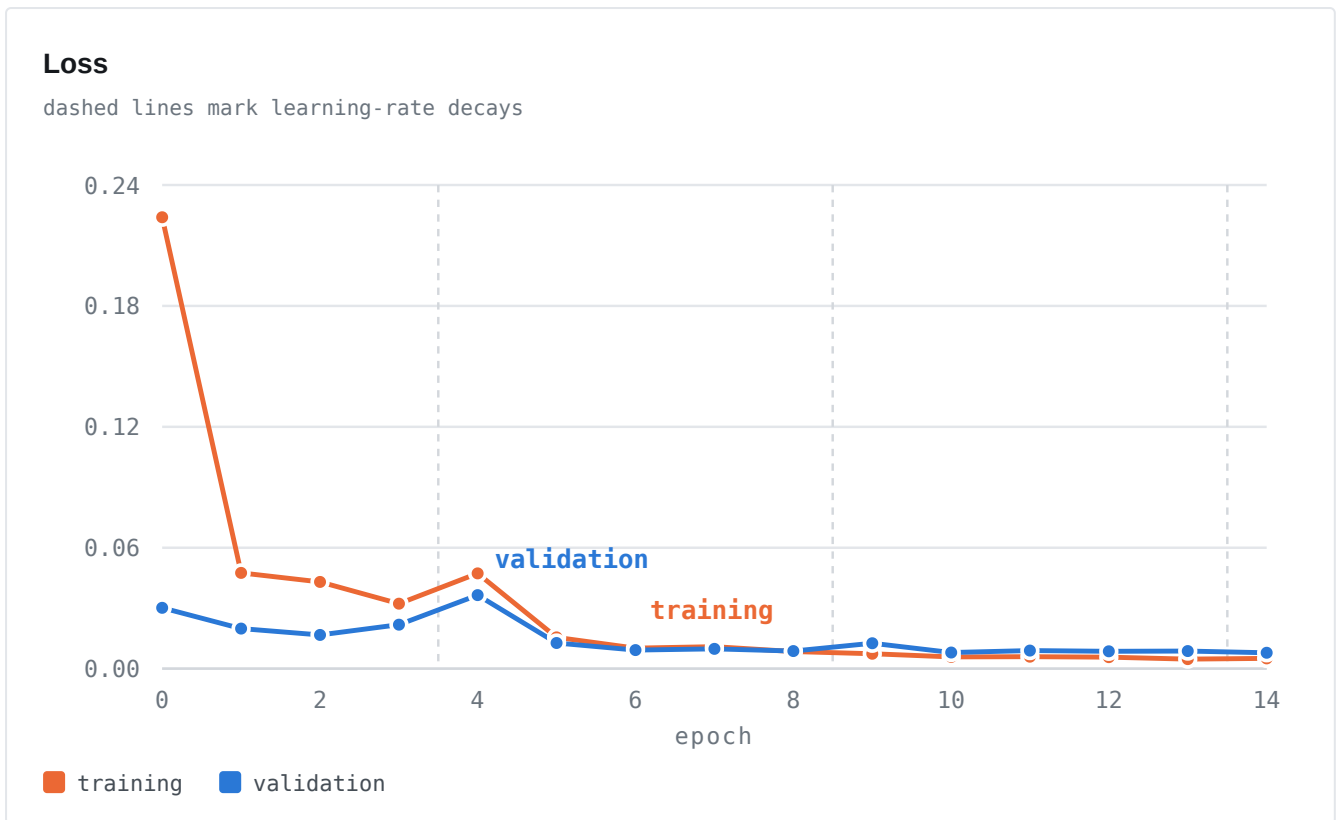
rows = true class • columns = predicted class

Precision on the defect class is a perfect 1.000 — every casting the model rejected really was defective. Recall is 0.9956: it found 451 of 453. For an inspection line those two numbers describe completely different risks, and reporting only the 99.72% hides which one you have.

TRAINING

Converged by epoch 6. The

remaining nine bought nothing.



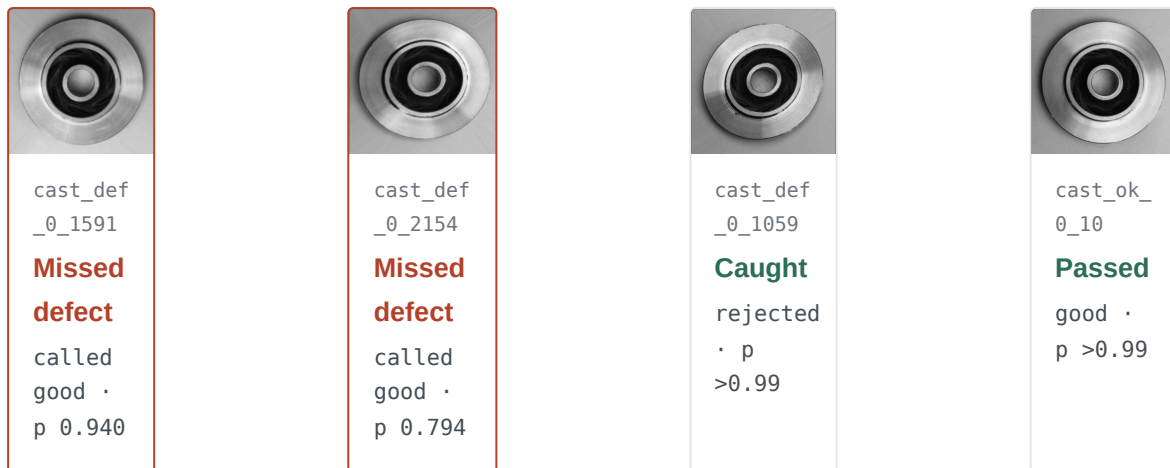
Epoch 0 alone reached 99.30%. That is the pretrained ImageNet features doing the work — the model was not learning vision from scratch, it was specialising what it already knew.

The wobble at epoch 4 is the interesting part: validation loss climbed to 0.0365, the worst of the run, and accuracy fell to 98.46%. The first learning-rate decay landed there and the run settled immediately afterwards. Without the scheduler that instability had no reason to stop — which matters more than usual here, because this library always saves the *final* epoch's weights rather than the best epoch's.

EPOCH	TRAIN LOSS	VAL LOSS	VAL ACCURACY
0	0.2240	0.0302	99.30%
1	0.0476	0.0199	99.44%
2	0.0430	0.0167	99.58%
3	0.0322	0.0218	99.58%
4	0.0473	0.0365	98.46%
5	0.0155	0.0128	99.58%
6	0.0101	0.0093	99.72%
7	0.0110	0.0098	99.72%
8	0.0084	0.0087	99.72%
9	0.0074	0.0126	99.44%
10	0.0059	0.0079	99.72%
11	0.0059	0.0090	99.72%
12	0.0057	0.0086	99.72%
13	0.0048	0.0088	99.72%
14	0.0051	0.0078	99.72%

THE TWO MISSES

Neither is an obvious mistake.



At this resolution the two misses read as clean parts to the eye as well. The caught defect on the right shows visible irregularity around the rim; the missed pair do not, at least not at 300×300 after the greyscale-and-resize the pipeline applies. Whatever marks them defective is subtle, possibly outside the frame, or arguably a labelling edge case.

Critically, **the model was never confidently wrong**. Across all 715 predictions the mean confidence on correct calls is 0.9984, and only **five images** fall below 0.90. Its uncertainty is concentrated exactly where the genuine ambiguity is — which is what makes the next section possible.

CATCHING THEM

Not a training problem. A threshold decision.

The model emits a probability; anything above 0.50 is currently called defective. Lowering that bar costs nothing — no retraining, no GPU time, one number:

FLAG IF $P(\text{DEFECT}) \geq$		MISSED DEFECTS	FALSE ALARMS	ACCURACY
0.50	current	2	0	99.72%
0.20		1	1	99.72%
0.05	catches both	0	6	99.16%
0.01		0	11	98.46%

At 0.05 **zero defects escape**, at the cost of scrapping six good castings. Which trade is right depends on economics rather than modelling: what does a defective casting reaching a customer cost, against scrapping a good one? For most safety-relevant parts, six scraps to prevent two escapes is an easy call.

The alternative framing is a review queue. Route anything below 0.95 confidence to a human inspector and you would catch both misses while flagging roughly five pieces out of 715 — under 1% of throughput — for a second look.

Two errors is statistically indistinguishable from zero. With 715 samples, the 95% confidence interval on 99.72% runs from 99.33% to 100% — anywhere between 0 and 5 errors is consistent with the same underlying model. Re-run with a different random seed and the count may well change without anything else having.

So resist tuning hyperparameters to chase these two. Any "improvement" that moves the count by one or two is fitting noise, and it will not reproduce.

READING THIS HONESTLY

Three things to state plainly in the writeup.

This dataset is not hard. Published results on the casting set sit above 98%. 99.72% reflects a well-posed problem with consistent lighting and framing, not an exceptional model. The interesting contribution is the error analysis and the threshold policy, not the headline figure.

The validation set is the dataset's own test split. Reading the training curve off it and choosing settings accordingly makes the final number mildly optimistic. Standard practice at this scale, but worth one sentence of acknowledgement rather than silence.

No data augmentation was used. The pipeline applies greyscale conversion, a resize to 224×224 and nothing else — no flips, rotations or brightness variation. That is unusual, and it is the most likely source of real gains if a better number is wanted: augmentation first, a modern backbone such as ResNet-18 second, higher input resolution third. Another learning-rate sweep would be the least productive of the four.

THE DATA BEHIND THIS PAGE

Every figure here came out of one JSON file.

The run wrote `results/alexnet-20260914-181039/` containing `results.json` (full detail — run metadata, hyperparameters, per-epoch history, per-class metrics, confusion matrix, inspection summary, and all 715 individual predictions with confidences), `summary.csv` (one row per epoch) and `predictions.csv` (one row per validation image).

Nothing on this page was transcribed by hand, and every number is reproducible from that directory. Each run writes its own timestamped folder, so runs accumulate rather than overwrite — which is what makes comparing a ResNet run against this one straightforward later.

gfitz01/Electronics-Manufacturing-Capstone-26-27 · branch modernization
AlexNet (ImageNet pretrained) · 6,633 train / 715 validation · 15 epochs · batch 16 · Adam · lr 1e-4 · StepLR step 5 gamma 0.1
Source: results/alexnet-20260914-181039/results.json